

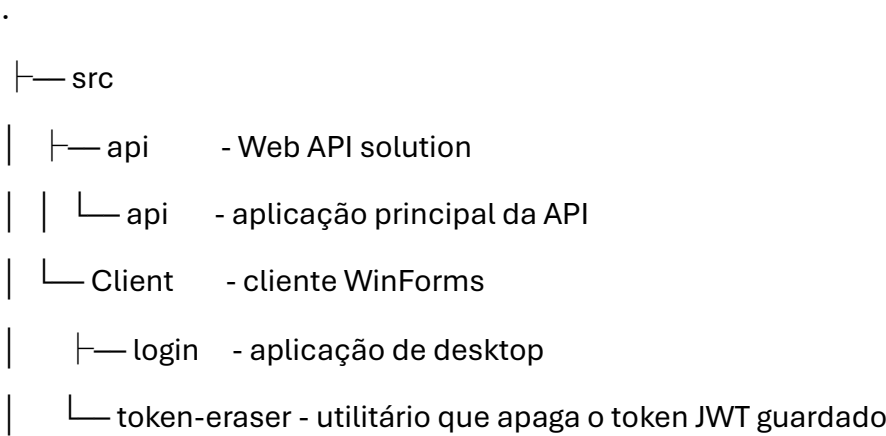
Manual do Programador

Aplicação: FinSync (PSI1623R_TiagoOliveira_2223234)
Tipo: API ASP.NET Core + Cliente WinForms
Autor: Tiago Oliveira
Ano Letivo: 2023/2024

Contents

- 1. Estrutura do Repositório 1
- 2. Pré-requisitos 2
- 3. Configuração Inicial 2
 - API 2
 - Cliente WinForms 3
- 4. Estrutura Técnica (Resumo) 3
 - Cliente WinForms 3
- 5. Notas Técnicas Adicionais 4
- 6. Endpoints e Funcionalidades 4
- 7. Debugging e Desenvolvimento 7

1. Estrutura do Repositório



└ docs - documentação técnica e académica

A API usa **.NET 8** e o cliente utiliza **.NET Framework 4.7.2**. A configuração da ligação à base de dados encontra-se em `appsettings.json`.

2. Pré-requisitos

- Visual Studio 2022 com suporte para .NET 8 e .NET Framework 4.7.2
- SQL Server (Express ou superior)
- SDK do .NET 8
- Dotnet EF
- Bibliotecas Guna (ambas encontram – se no `src/framework`):
 - `Guna.UI2.WinForms.dll`
 - `Guna.UI.WinForms.dll`

Estas duas DLLs devem ser adicionadas manualmente como referências ao projecto **WinForms** para garantir a renderização correta dos componentes gráficos modernos.

3. Configuração Inicial

API

1. Clonar o repositório.
2. Navegar até `src/api/api`.
3. Criar uma cópia do `appsettings.Development.json` com o nome `appsettings.json`. (opcional)
4. Alterar a string de conexão `ConnectionStrings:DefaultConnection` para apontar para a instância correta do SQL Server.
5. No terminal, executar:

```
dotnet ef database update

./starter.bat
```
6. A API estará disponível em `http://localhost:5034`.

7. Swagger está acessível via <http://localhost:5034/swagger> (modo de desenvolvimento).
8. Para testar a API referir ao [Ponto 6](#)

Cliente WinForms

1. Abrir o ficheiro de solução src/Client/login.sln no Visual Studio.
2. Adicionar as referências manualmente às bibliotecas Guna indicadas.
3. Compilar e correr a aplicação. Esta comunica com a API e depende de um ficheiro auth.token gerado após login.

Para apagar este token e forçar nova autenticação, pode-se correr o utilitário token-eraser.

4. Estrutura Técnica (Resumo)

API

- Controllers/ – endpoints REST (Autenticação, Rendimentos, Despesas, Categorias, Objectivos)
- Models/ – entidades de base de dados (User, Income, Expense, Goal, etc.)
- DTOs/ – modelos de dados de entrada/saída
- Jobs/RecurringIncomeJob.cs – rendimentos recorrentes automáticos via Quartz
- Panel/Panel.cs – menu interativo de administração no terminal
- Utils/ – hashing de palavras-passe, validação de JWT, geração de tokens
- starter.bat – reinicia a API automaticamente se sair com código 100

Cliente WinForms

- Program.cs – entrada principal, armazena e gere o JWT
 - Helpers/ – chamadas HTTP, gestão de sessão
 - Tabs/ – separadores gráficos: Incomes, Expenses, Goals
 - token-eraser/ – app de consola para remover ficheiro auth.token
-

5. Notas Técnicas Adicionais

- O auth.token é um ficheiro de texto criado após login com o token JWT. Este é incluído em todas as chamadas subsequentes via Authorization: Bearer <token>.
- A API exige sempre o header x-api-key: 12345-abcdef-67890 (definido em appsettings.json > ApiKeySettings).
- Toda a lógica de autenticação e autorização está baseada em JWT e roles (user, admin).
- As palavras-passe são guardadas com hashing PBKDF2 (PasswordHelper.cs).

6. Endpoints e Funcionalidades

AuthController (Autenticação)

Método	Endpoint	Descrição
POST	/api/auth/login	Fazer login e receber token JWT
POST	/api/auth/register	Registar um novo utilizador
POST	/api/auth/reset-password	Redefinir a palavra-passe do utilizador

AdminController (Requer papel de administrador)

Método	Endpoint	Descrição
GET	/api/admin/users	Listar todos os utilizadores
POST	/api/admin/promote/{id}	Promover um utilizador a administrador

BalanceController (Saldo)

Método	Endpoint	Descrição
GET	/api/balance/monthly	Obter saldo do mês atual
GET	/api/balance	Obter saldo total

BudgetController (Orçamento)

Método	Endpoint	Descrição
POST	/api/budget	Criar um novo orçamento
GET	/api/budget	Listar todos os orçamentos
PUT	/api/budget/{id}	Atualizar um orçamento
DELETE	/api/budget/{id}	Apagar um orçamento

CategoryController (Categorias)

Método	Endpoint	Descrição
GET	/api/category	Listar todas as categorias
GET	/api/category/{id}	Obter categoria por ID
POST	/api/category	Criar uma nova categoria
PUT	/api/category/{id}	Atualizar uma categoria
DELETE	/api/category/{id}	Apagar uma categoria

ExpenseController (Despesas)

Método	Endpoint	Descrição
POST	/api/expense	Criar uma nova despesa
GET	/api/expense	Listar todas as despesas
GET	/api/expense/summary?period=monthly	Obter resumo das despesas (diário/semanal/mensal/anual)
GET	/api/expense/{id}	Obter uma despesa por ID

Método	Endpoint	Descrição
PUT	/api/expense/{id}	Atualizar uma despesa
DELETE	/api/expense/{id}	Apagar uma despesa

GoalController (Objetivos)

Método	Endpoint	Descrição
POST	/api/goal	Criar um novo objetivo
GET	/api/goal	Listar todos os objetivos
POST	/api/goal/{id}/save	Adicionar poupança a um objetivo
PUT	/api/goal/{id}	Atualizar um objetivo
DELETE	/api/goal/{id}	Apagar um objetivo

IncomeController (Receitas)

Método	Endpoint	Descrição
POST	/api/income	Criar uma receita (suporta recorrência)
GET	/api/income	Listar todas as receitas
GET	/api/income/summary?period=monthly	Obter resumo das receitas
GET	/api/income/non-recurring	Listar receitas não recorrentes
GET	/api/income/{id}	Obter uma receita por ID
DELETE	/api/income/{id}	Apagar uma receita
PUT	/api/income/{id}	Atualizar uma receita (suporta recorrência)

HealthController (Estado do Sistema)

Método	Endpoint	Descrição
GET	/health	Verificar o estado da API

7. Debugging e Desenvolvimento

- Usar Swagger (/swagger) para testar endpoints.
- Validar se o token JWT não expirou.
- Verificar ficheiro de logs ou terminal para mensagens de erro.
- Assegurar que a string de conexão está correta em appsettings.json.
- Para testes rápidos, pode-se promover manualmente utilizadores a admin na base de dados.